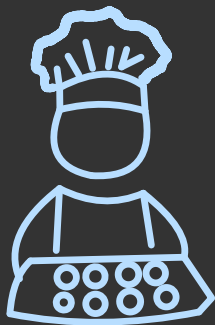
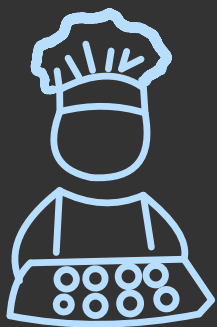


functions and arguments

CSCI
134



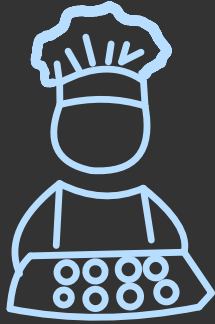
primitive expressions refer to
concrete, atomic things



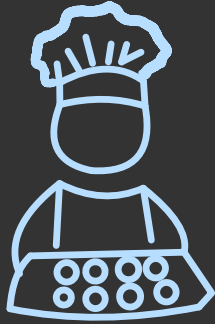
me
cookie



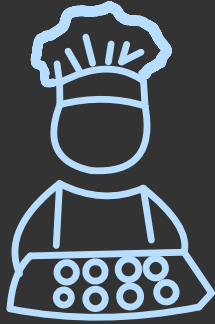
and make sense on their own



the word **give** operates a bit differently



its meaning is typically **incomplete** on its own



give what?

give whom a
cookie?

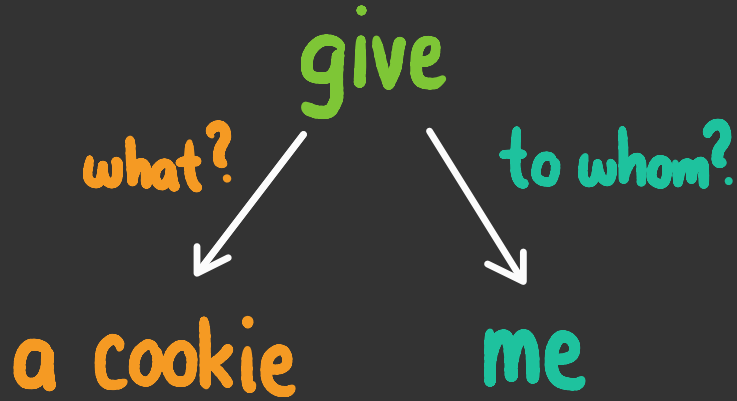
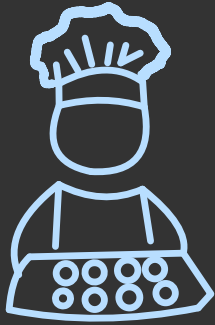
give

give a
cookie

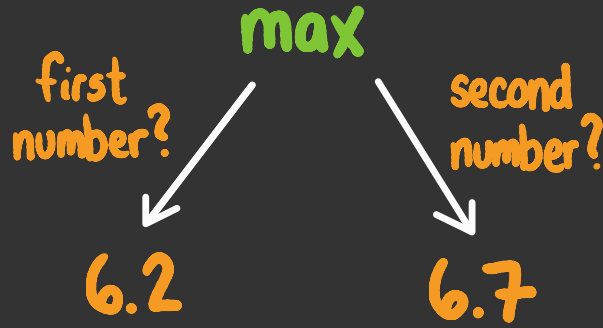
give me a
cookie



its meaning is typically incomplete on its own



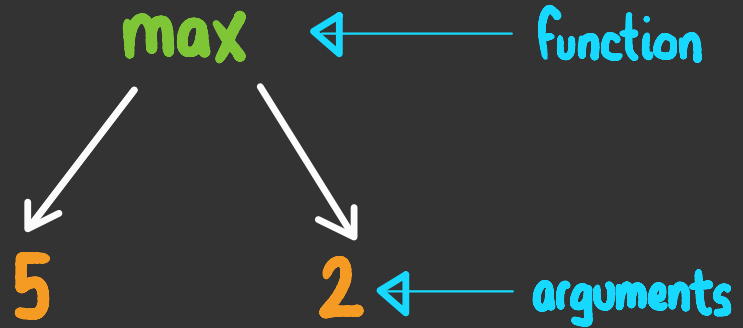
to have a complete meaning,
we have to specify its arguments



computers prefer to talk about numbers,
but the same ideas apply



these "verbs" are called
functions



these "verbs" are called
functions



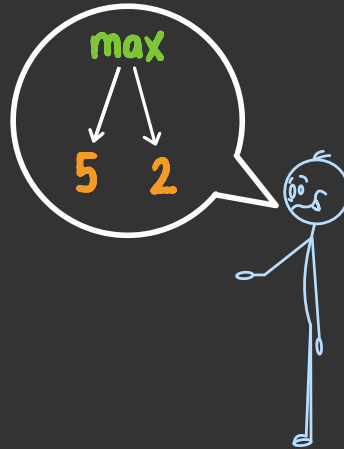
`abs` ← function
↓
`-7.2` ← argument

not all functions have **two** arguments:
some have **more than two**, some have **one**, some have **zero**

presenting **SIX** useful
python functions !

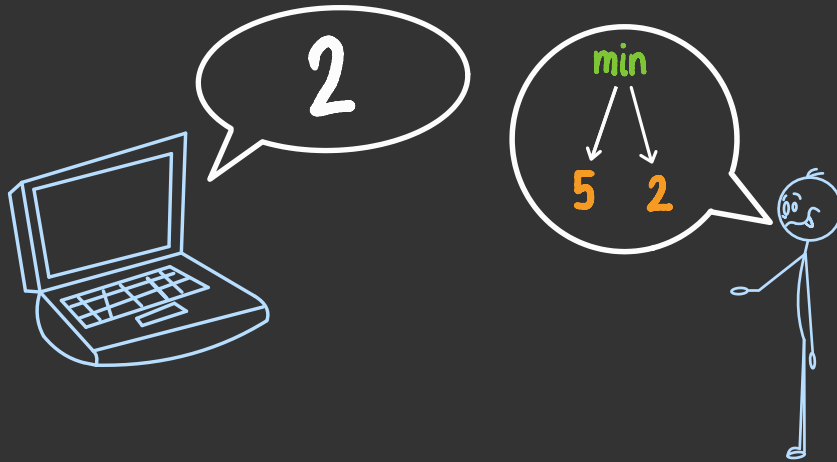
the **max** function computes
the maximum of **its two arguments**

e.g. the value of $\begin{array}{c} \text{max} \\ \swarrow \searrow \\ 5 \quad 2 \end{array}$ is 5



the **min** function computes
the minimum of **its two arguments**

e.g. the value of $\min \begin{matrix} \swarrow \searrow \\ 5 \quad 2 \end{matrix}$ is 2



the **pow** function computes
its first argument to the power of its second

e.g. the value of $\text{pow}(5, 2)$ is 25



the **abs** function computes
the absolute value of **its argument**

e.g. the value of $\begin{array}{c} \text{abs} \\ \downarrow \\ -7.2 \end{array}$ is 7.2



the **round** function computes
the nearest integer to **its argument**

e.g. the value of **round**
↓
7.8 is 8

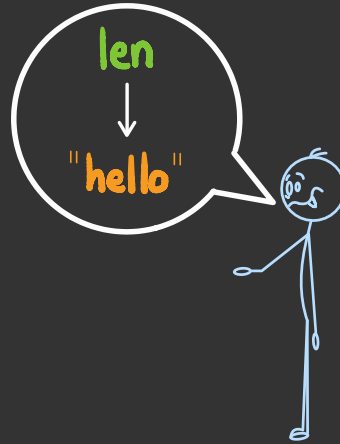


the **len** function computes
the length of **its (str) argument**

e.g. the value of **len**
↓
"hello" is 5



5



in addition, it is useful to know about
typecasting functions

typecasting functions

int (**expression**)

str (**expression**)

float (**expression**)

```
In [1]: int("3")  
Out[1]: 3
```

```
In [2]: int(3.67)  
Out[2]: 3
```

```
In [3]: str(3)  
Out[3]: '3'
```

```
In [4]: str(3.67)  
Out[4]: '3.67'
```

```
In [5]: float("3.67")  
Out[5]: 3.67
```

```
In [6]: float(3)  
Out[6]: 3.0
```

these functions convert expressions
from one type to another